

NSshard: Low-Cross-Shard Sharding via Account Partitioning for Blockchain-Based IoT

Bo Yin^{1b}, *Member, IEEE*, Qianwen Xie, Rongwei Xu, and Ke Gu^{1b}, *Member, IEEE*

Abstract—The Internet of Things (IoT) links the physical world to computing systems, and blockchain presents an opportunity to address the issues of weak interoperability and security flaws within IoT. However, blockchain faces the challenge of low throughput and scalability. Sharding is a promising solution, but it divides the blockchain into multiple committees, making the attack cost of malicious nodes lower. Sharding also leads to a large number of cross-committee transactions, which degrades the system's performance. In this article, we propose the NSshard sharding framework that provides secure and low-cross-committee scaling. NSshard consists of network sharding and state sharding. We first propose a reputation score-based network sharding, which assigns each node a reputation score to reward its honest verification of transactions and penalizes its malicious behavior. This network sharding uses a random but balanced distribution of reputation scores, thereby decreasing the risk of collusion. We also propose a graph-based account partitioning scheme for state partitioning. To reduce the amount of cross-committee transactions, the scheme uses an undirected weighted graph to depict accounts and transactions. We design two algorithms based on edge splitting and overlapping community discovery, respectively. We also propose a dynamic sharding method to handle new transactions. We conduct extensive experiments to evaluate the efficiency of the proposed framework based on Ethereum transaction data. The experimental results show that our proposed framework can reduce the number of cross-committee transactions by 34.8% at 128 committees compared to the Metis algorithm.

Index Terms—Blockchain, efficient communications and networking, network architecture, sharding.

I. INTRODUCTION

INTERNET of Things (IoT) plays a significant part in linking the physical world to computing systems [1], [2]. However, due to its decentralized and diverse nature, IoT faces issues with weak interoperability and security flaws [3], [4]. The emergence of blockchain technologies presents an opportunity to address these issues. Blockchain is a distributed ledger maintained by all nodes in the network, with the characteristics of decentralization, tamper resistance, and data transparency [5]. This innovative technology has been

widely utilized in finance [6], [7], energy [8], [9], and healthcare [10], [11]. Nevertheless, each node in the blockchain needs to store and verify all transactions, cutting down on transaction commit latency. It is difficult to achieve decentralization while retaining high security and scalability, which is an ongoing challenge for blockchain technology. For example, Ethereum processes 13 transactions per second with a 5-min confirmation latency, compared to ten transactions per second by Bitcoin with a 144-min confirmation delay.

Sharding is an effective and promising approach to expanding the system's throughput horizontally as the number of participants increases [12]. It was initially used in the field of databases for managing and distributing large-scale datasets [13]. In the blockchain field, this concept involves dividing the blockchain network into multiple committees, each called a "shard." Each committee processes different transactions in parallel, and each node is only responsible for storing partial states and handling transactions of its shard. This helps reduce the storage and processing burden on individual nodes, leading to increased transaction throughput. However, sharding also presents challenges. When a blockchain network is divided into multiple committees, its security decreases. With smaller committees, malicious nodes can more easily launch attacks. For PoW consensus protocol, if the number of malicious nodes in the shard exceeds 51%, the shard can be easily controlled. Additionally, cross-committee transactions are a notable issue. In sharded blockchain networks, a significant portion of transactions occur between different committees. Compared to intracommittee transactions, handling cross-committee transactions involves executing a cross-committee protocol to achieve consensus, which increases communication costs and computational burden [14].

Current blockchain sharding technologies can be categorized into three categories: 1) network sharding; 2) transaction sharding; and 3) state sharding [15]. Network sharding, focusing on node partitioning, is the basis of transaction sharding and state sharding. Transaction sharding involves transaction partitioning, with each committee only processing its assigned transactions. State sharding divides the world state into several parts, with different committees storing different states, rather than the complete world state. Prior network sharding methods mostly perform random division of nodes. For example, Omniledger [16] uses the bias-resistant public-randomness protocol RandHound [17] and verifiable random function (VRF) [18] to assign nodes to various committees. However, this random sharding method does not take into account node

Received 27 July 2024; revised 26 October 2024; accepted 19 November 2024. Date of publication 25 February 2025; date of current version 26 March 2025. This work was supported in part by the Changsha Natural Science Foundation of China under Grant kq2402030, and in part by the Natural Science Foundation of Hunan Province of China under Grant 2024JJ5043. (Corresponding author: Bo Yin.)

The authors are with the School of Computer and Communication Engineering, Changsha University of Science and Technology, Changsha 410114, China (e-mail: yinbo@hnu.edu.cn; lucky-xqw@stu.csust.edu.cn; xrwsmc@stu.csust.edu.cn; gk4572@163.com).

Digital Object Identifier 10.1109/JIOT.2024.3506593

characteristics, such as computing power or historical behavior. Prior transaction sharding approaches [16], [19], [20] relied on hash mapping to allocate transactions, often resulting in around 90% of cross-committee transactions due to a lack of consideration for transaction relationships [21]. There are also state-sharding approaches [16], [22], [23], [24]. State sharding turns cross-committee transactions into intracommittee transactions, allowing transactions to be conducted within a single committee. However, state sharding causes frequent state exchanges, making difficult to maintain world states. Moreover, state sharding can easily lead to imbalanced workloads among committees. Some committees become hot ones; as a result, the confirmation time for transactions will be long.

In this article, we introduce a new sharding framework called NSshard, designed for secure and efficient blockchain scaling. This framework focuses on minimizing cross-committee transactions while ensuring system security and simplifying world-state management. To achieve security, we propose reputation score-based network sharding. Each blockchain node is assigned a reputation score, which rewards honest behavior and penalizes dishonest behavior. The network sharding groups nodes to ensure a random but balanced distribution of reputation scores, reducing the impact of malicious nodes. We also propose a graph-based account partitioning scheme to decrease cross-committee transactions. It presents accounts and transactions as an undirected weighted graph. The goal of the scheme is to minimize the total weight of edges across committees. This scheme helps manage the world state by minimizing vertex splitting and maintaining workload balance by imposing a capacity limit on each shard. To implement these concepts, we introduce two algorithms: 1) the maximum edge weight (MEW) algorithm and 2) the overlapping community-based (OC) algorithm. MEW uses edge splitting for state sharding and designs a segmentation function to determine the need for node/account splitting. The OC algorithm leverages the concept of overlapping communities to group tightly connected accounts within the same committee, thus supporting state sharding. Committees are allowed to overlap. The OC algorithm incorporates edge weight into community discovery, generating a fixed number of committees through two stages. In addition, we design dynamic sharding approaches to handle new transactions.

The main contributions of this article are as follows.

- 1) We propose the NSshard sharding framework that provides low-cross-committee scaling while improving the system's security. NSshard contains network sharding and state sharding. The former reduces the conspiracy of malicious nodes; the latter decreases the amount of cross-committee transactions while simplifying world-state management.
- 2) We propose a reputation score-based network sharding. We define the reputation score function that rewards nodes for their honest verification of transactions and penalizes them for their malicious behavior. This network sharding prevents malicious nodes from congregating in specific shards.

- 3) We propose the MEW and OC algorithms for state sharding. An undirected weighted graph is used to depict accounts and transactions. The MEW algorithm is based on edge splitting, while the OC algorithm is based on overlapping community discovery. We also achieve dynamic sharding when new transactions emerge at a low cost.
- 4) We used real Ethereum transaction data for experimental evaluation. We tested the total weight of cross-committee transactions, number of account segments, workload balance, system throughput, and transaction latency. The experimental results showed that the balance of the distribution of malicious nodes is promoted by 50% at 64 committees. At 128 committees, the OC algorithm and MEW algorithm produce 53.3% and 34.8% less cross-committee transaction weight than the Metis [25] algorithm, respectively.

The remainder of this article is structured as follows. Section II summarizes the related work on blockchain sharding. Section III introduces the network sharding based on the reputation score. Section IV introduces our proposed MEW and OC algorithms. Dynamic sharding and account management are also discussed. Section V discusses the experimental results. Finally, Section VI summarizes this article.

II. RELATED WORK

Elastico [19] was the first to apply the sharding concept to blockchain. It divides the entire blockchain network into a set of committees, each handling different transactions. It almost linearly increases the transaction throughput and computing power of blockchain networks. RapidChain [20] is a fast consensus protocol that supports block pipelining and large block gossiping. It is resistant to Byzantine faults caused by up to one-third of its participants. Omniledger [16] and Monoxide [22] are state-sharding schemes where consensus nodes only need to save local committee data and part of the global data for cross-committee transaction processing. These methods prevent attackers from colluding with each other by periodically changing network configurations. SSChain [26] utilizes market incentivization that allows nodes to freely join one or more committees without periodically changing the network configuration, thereby eliminating the overhead of heavy data migration. SkyChain [12] is the first framework based on dynamic sharding. It proposes an adaptive ledger protocol and a deep reinforcement learning-based sharding approach, which dynamically adjusts the resharding interval, number of committees, and block size.

The uneven distribution of transactions among committees can lead to the "hot-committee" issue, causing delays in confirming transactions. To solve this problem, BrokerChain [23] utilizes fine-grained state partitioning and account segmentation. It uses the Metis [25] algorithm for account partitioning to balance the load among committees and employs a broker account mechanism to manage cross-committee transactions. GridB [27] employs a greedy planning strategy to allocate workload to committees, as it does in distributed databases. It allows off-chain data migration. LB-Chain [28] predicts

TABLE I
COMPARISON OF NSSHARD WITH EXISTING SHARDING APPROACHES

	TX model	Sharding method	# nodes	malicious node fraction	Cross-committee TX ratio
Elastico [19]	UTXO	Random	1600	1/4	No
Omniledger [16]	UTXO	Random	1800	1/4	93.5%
RapidChain [20]	UTXO	Simple active/inactive	4000	1/3	96.3%
Monoxide [22]	Account	Modular sharding	1000	1/2	98.6%
SSChain [26]	UTXO	Market incentive mechanism	1800	1/4	N/A
Pyramid [29]	Account	Layered sharding	1000	1/3	N/A
Brokerchain [23]	Account	Metis	N/A	N/A	72.4%
LB-Chain [28]	Account	Long short-term memory model	256	1/4	N/A
Optchain [14]	UTXO	PageRank	N/A	N/A	N/A
X-shard [33]	Account	Metis	N/A	1/3	20%
NSshard	Account	Reputation score and graph-based sharding	1000	1/3	17%

the number of incoming account transactions and uses this information to determine which accounts should be assigned to which committees. To ensure a balanced transaction distribution, LB-Chain periodically transfers active accounts from heavily loaded committees to less-loaded committees.

Several approaches have been proposed to enhance the efficiency of cross-committee transaction processing. One such method is Pyramid [29], which is a layered sharding blockchain system. This system allows some committees to store records from multiple committees, enabling the processing of cross-committee transactions within these committees. However, this approach comes with a tradeoff as it reduces the system's security. Cherubim [30] is a general pipelined two-phase commit (P-2PC) framework for parallel processing of the preparation and commit of unrelated transactions. However, it does not consider the workload imbalance issue. Li et al. [31] embedded lightweight global consensus into parallel intracommittee consensus to achieve the instant atomicity of cross-committee transactions. They also proposed a dynamic sharding management mechanism to handle transaction congestion more efficiently. However, this method prioritizes system security and works well when the committee contains a small number of nodes. To minimize these transactions, Optchain [14] considers transactions as vertices in an online graph. It utilizes a lightweight and on-the-fly transaction placement method to allocate related transactions into the same committees. Estuary [24] is a mechanism for splitting and aggregating states, allowing transactions to be executed within a single committee. Zhang et al. [32] approached the transaction allocation problem as a community discovery problem in graph theory. They proposed a dynamic allocation mechanism called TxAllo based on the account/balance model, which optimizes system throughput and reduces cross-committee transactions. X-shard [33] is a transaction allocation method based on historical transaction patterns. It groups frequently interacting accounts into the same committee. Additionally, it proposes a strategy to process cross-committee transactions with minimal delay.

Compared with existing work, our research focuses on how to implement network sharding to improve system security and account partitioning to reduce cross-committee transactions on the account/balance-based model. Although many approaches [16], [20], [22] use periodic reorganization of committees to improve system security, this

reconfiguration-based approach can result in significant state migration. Optchain and Estuary reduce cross-committee transactions, but they focus on the unspent transaction output (UTXO) model and cannot be used for the account/balance model. Both BrokerChain and X-shard use the Metis algorithm for sharding. However, our experiments have shown that our MEW and OC algorithms generate lower cross-committee transaction weights than the Metis algorithm. Although TxAllo employs the idea of community discovery for sharding, it utilizes a nonoverlapping community discovery algorithm Louvain [34]. An account can only be allocated to one committee, which is not suitable for complex scenarios where accounts have complex connections. Differently, our OC algorithm uses overlapping community discovery to support state sharding. Table I shows the comparison of our proposed NSshard with existing sharding approaches where TX is the abbreviation of transaction.

III. NETWORK SHARDING

A. System Model

Most existing network sharding approaches ensure system security by randomly partitioning nodes and periodically repartitioning them. However, nodes' historical behavior, such as malicious validation of transactions, was not considered. Therefore, we introduce the concept of reputation score to capture the above characteristics.

There are two types of committees in our sharded system: one council committee (C-committee) and other general committees (G-committees). G-committee is responsible for generating transaction block (TXB) and transaction validation block (TXVB). The TXVB contains transaction verification records of nodes, which are used to determine whether a node honestly verifies transactions, as represented by their reputation score. The C-committee is responsible for computing the reputation scores of all nodes and partitioning account states. This information is packaged into the state reputation block (SRB). Both the G-committee and the C-committee use the Byzantine fault tolerance (PBFT) consensus protocol for intracommittee consensus. To reduce communication overhead among committees, the system selects a leader node in each committee based on the reputation score. The leader node manages the nodes in the committee, drives the

TABLE II
SUMMARY OF NOTATIONS

Notation	Description
N	set of nodes
n_i	a node in N
S	set of committees
S_i	a committee in S
r_i^e	the reputation score of n_i in epoch e
LC_i	set of leader node candidate of S_i
$N(v)$	set of neighboring nodes of vertex v
(v, u)	edge between vertex v and vertex u
$w(v, u)$	weight of (v, u)
w_v	total number of transactions involving v
C	capacity limit of a committee
δ	balancing factor
$F(S_i, v)$	segmentation function of v in S_i
α	segmentation factor
LS_v	set of labels for vertex v
$B(S_i, v)$	belonging coefficient of v in S_i
$M(S_i, S_j)$	merging coefficient of S_i and S_j

consensus protocol, and communicates with other committees [16], [20], [35]. Any node can join the system by solving a proof-of-work problem but will start working in the next epoch. Therefore, the number of nodes in an epoch is fixed.

At the end of each epoch, the C-committee reads the TXBs of G-committees to update the account transaction graph and run the account partitioning algorithm. Meanwhile, the C-committee calculates the reputation scores of all nodes based on TXBs and TXVBs. It broadcasts the sharding results and reputation scores in the network using SRBs. In this way, every node knows all nodes' reputation scores, which will incentivize nodes to honestly verify transactions. Table II lists the notations that are frequently used in this article.

B. Network Sharding and Leader Node Election

Assume that there are n nodes $N = \{n_1, n_2, \dots, n_n\}$ and $k + 1$ committees $S = \{S_1, S_2, \dots, S_k, S_{k+1}\}$, where the first k committees are G-committee and the $k + 1$ th committee is C-committee. Network sharding involves allocating nodes into $k + 1$ committees and selecting a leader node. For the security of the sharded system, it is necessary to ensure the randomness of nodes and an even distribution of reputation scores between committees. "Even distribution" means that the sum of reputation scores and the number of nodes in each committee do not differ much. Ideally, the two numbers are the same for different committees. Guaranteed randomness ensures the unpredictability of the entire sharding process, such as predicting which committee a node will belong to and which node will be chosen as the leader.

Network Sharding: We use r_i to represent the reputation score of node $n_i \in N$. This score is based on the node's historical behavior. The reputation score of node n_i at epoch $e - 1$ is denoted as r_i^{e-1} . The set of reputation scores of all nodes at epoch $e - 1$ is denoted as $R^{e-1} = \{r_1^{e-1}, r_2^{e-1}, \dots, r_n^{e-1}\}$. Algorithm 1 provides the pseudo code for network sharding at epoch e . The algorithm iteratively selects the node n_i with the highest reputation score for sharding (line 5). To ensure balanced node amounts and reputation scores among committees,

Algorithm 1: Network Sharding

Input: $seed^e, R^{e-1} = \{r_1^{e-1}, r_2^{e-1}, \dots, r_n^{e-1}\}$.
Output: $S = \{S_1, S_2, \dots, S_k, S_{k+1}\}, L = \{L_1, L_2, \dots, L_k, L_{k+1}\}$.

```

1 Procedure NodeSharding( $seed^e, R^{e-1}$ ):
2   Initialization:  $S_j \leftarrow \emptyset, L_j \leftarrow \emptyset$  for each  $1 \leq j \leq k + 1$ ;
3   Generating random numbers  $seed^e$  and  $\omega$ ;
4   while  $R^{e-1} \neq \emptyset$  do
5     Picking out  $r_i^{e-1}$  with the maximum value in  $R^{e-1}$ ;
6     if  $S_j$  is the only committee with the minimum number of nodes
7       then
8          $S_j \leftarrow S_j \cup n_i$ ;
9     else if there are multiple  $S_j$  then
10      if  $S_j$  has minimum reputataion score then
11         $S_j \leftarrow S_j \cup n_i$ ;
12      else
13         $S_j \leftarrow S_j \cup n_i$  with  $j = RNG \bmod y$ ;
14 Procedure LeaderSelection( $seed^e, S$ ):
15   for each  $S_j \in S$  do
16     for each  $n_i \in S_j$  do
17       if  $r_i^{e-1} \geq \bar{r}_j$  then
18          $LC_j \leftarrow LC_j \cup n_i, rp_i = \omega / r_i^{e-1}$ ;
19       choose  $n_i \in LC_j$  with the minimum  $rp_i$ ;
20        $L_j \leftarrow n_i$ ;

```

it partitions node n_i preferentially into the committee with the fewest nodes (line 7). If multiple committees have the same number of nodes, the algorithm partitions node n_i into the committee with the lowest sum of reputation scores (lines 9 and 10). If these committees have the same sum of reputation scores, one of these committees is chosen randomly (line 12). To support this random partitioning, we generate a random integer RNG using the block hash value of the previous SRB as a random seed $seed^e$ for epoch e .

Leader Node: The leader node is elected based on randomness and the reputation scores of the nodes in the committee. Only nodes with high-reputation scores are more likely to be elected as leader nodes. Specifically, given a committee S_j , $\bar{r}_j$ represents the average reputation score of all nodes in the committee. We add nodes with a reputation score higher than $\bar{r}_j$ to the candidate set LC_j (lines 16 and 17, Algorithm 1). To ensure randomness, a random number $\omega \in (0, 1)$ is generated based on $seed^e$. For each node in LC_i , ω is divided by the reputation score (line 17, Algorithm 1). Obviously, the smaller of ω / r_i^{e-1} the better. After that, the node with the minimum value is elected as the leader node L_j of committee S_j (lines 18 and 19, Algorithm 1).

Reputation Score: The behavior of a node when verifying a transaction can fall into one of three categories: 1) honest validation; 2) malicious validation; or 3) no validation. Malicious validation involves disagreeing with valid transactions and agreeing with invalid ones, and nodes engaging in this behavior should face punishments. Additionally, to prevent highly reputable nodes from controlling the sharding process, the growth of reputation scores for such nodes should be slowed down when there is a significant difference in their scores. Conversely, the growth of reputation scores can be accelerated when the difference is small. The difference in node reputation

scores is measured using the standard deviation σ , where a small value of σ indicates a small difference and the opposite indicates a large difference. Therefore, the reputation score of node n_i in epoch e is calculated as follows:

$$r_i^e = r_i^{e-1} + \tau_1 \sum_{j=1}^{\eta} \frac{\tau_2^j}{\sigma} \ln(r_i^{e-1} + 1) \quad (1)$$

where

$$\sigma = \sqrt{\sum_{i=1}^n (r_i^{e-1} - \bar{r}^{e-1})^2 / n}. \quad (2)$$

Here, η represents the total number of transactions in the committee where node n_i is located in epoch $e-1$; $\bar{r}^{e-1}$ is the average reputation score of all nodes during the epoch $e-1$. In the epoch e , if a node performs malicious behaviors, we set $\tau_1 = -1$; otherwise, $\tau_1 = 1$. If a node does not perform verification on a transaction, we set $\tau_2^j = 0$; otherwise, $\tau_2^j = 1$.

IV. ACCOUNT PARTITIONING

A. Problem Formulation

Accounts and transactions form an undirected weighted graph: accounts as vertices and transactions as edges. The edge weight represents the number of transactions between accounts.

Definition 1 (Account Transaction Graph): We denote an account transaction graph as $G = (V, E, W)$ where V represents the set of vertices corresponding to accounts, $E \subseteq V \times V$ represents the set of edges corresponding to transactions between accounts, and W represents the set of weights of the edges. Let $N(v)$ represent the set of neighboring nodes of v , representing accounts trading with v . Let (v, u) represent the edge between vertices v and u . $w(v, u)$ is the weight of the edge (v, u) , representing the number of transactions between the two accounts. The weight w_v of vertex v refers to the total number of transactions involving v

$$w_v = \sum_{u \in N(v)} w(v, u). \quad (3)$$

Account partitioning in blockchain involves dividing the set V of accounts into k subsets or committees, denoted as $S = \{S_1, S_2, \dots, S_k\}$. In this context, we allow for state sharding, where an account can exist in multiple committees. We use the term ‘‘account segment’’ to refer to a segmented account in a committee. The world state of an account is the sum of the states of all account segments in multiple committees. The advantage of state sharding is that a transaction will have its sender account and receiver account in the same committee, thus converting cross-committee transactions to intracommittee transactions. However, it’s worth noting that the more committees an account exists in, the more difficult it becomes to maintain its world state.

Our objective is to minimize the amount of transactions between different committees while considering workload balance and world state maintenance.

Algorithm 2: MEW Algorithm

Input: $G, k, \delta, \alpha, \bar{w}$.
Output: $S = \{S_1, S_2, \dots, S_k\}$.

```

1 Procedure Sharding( $G, k, \delta$ ):
2    $i \leftarrow 0$ ;
3   while  $E \neq \emptyset$  do
4      $i \leftarrow i + 1$ ,  $S_i \leftarrow \emptyset$ ;
5      $e \leftarrow$  the edge with the maximum weight in  $G$ ;
6      $S_i \leftarrow S_i \cup e$ ;  $E \leftarrow E - (v, u)$ ;
7     while  $|S_i| < C$  do
8        $(v, u) \leftarrow$  the edge with the maximum weight where
9          $v \in S_i$  and  $u \in G$ ;
10       $S_i \leftarrow S_i \cup (v, u)$ ;
11       $E \leftarrow E - (v, u)$ ;
12      SegmentFunction( $S_i, \bar{w}, \alpha$ );
13 Procedure SegmentFunction( $S_i, \bar{w}, \alpha$ ):
14   for each  $v \in S_i$  do
15      $F(S_i, v) = \sum_{u \in N(v) - S_i} (w(v, u) - \alpha \bar{w})$ ;
16     if  $F(S_i, v) < 0$  then
17        $E \leftarrow E - \{(v, N(v) - S_i)\}$ ;

```

- 1) The weight of the edge between two accounts reflects the transaction amount. So, reducing the cross-committee transaction amount is the same as lowering the total weight of edges across committees.
- 2) We maintain workload balance between committees by imposing a capacity limit on the number of accounts in each committee. This limit, denoted as C , is $\delta(|V|/k)$, where $\delta \geq 1$ is the balancing factor. The number of accounts in a committee cannot exceed C .
- 3) To uphold the world state, we should reduce the total number of account segments.

We propose two account partitioning algorithms: 1) MEW algorithm and 2) OC algorithm. We also propose a dynamic sharding algorithm to handle the scenarios of new transactions and new accounts.

B. MEW Algorithm

To implement account partitioning, we utilize vertex splitting or edge splitting in the graph field. Vertex splitting involves grouping vertices into different committees, which does not support state sharding. On the other hand, edge partitioning groups edges into different committees. When an edge is partitioned to a committee, the two accounts involved are also partitioned to that committee. Because a vertex may be associated with multiple edges, edge partitioning allows a vertex to exist in multiple committees, thus supporting state sharding. Therefore, we propose the MEW algorithm that employs edge splitting for account sharding.

The goal of minimizing the total weight of the edges connecting different committees is equivalent to maximizing the total weight of intracommittee edges. Hence, to create a committee S_i , the MEW algorithm always selects the edge (v, u) with the highest weight and assigns it to S_i , thus adding both accounts v and u to S_i .

MEW constructs k committees sequentially, as shown in Algorithm 2. When constructing S_i , the algorithm considers the neighboring edges of S_i , i.e., the edges involving the

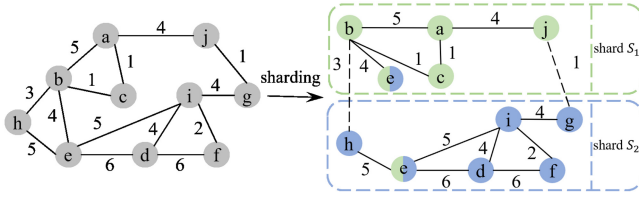


Fig. 1. Example of the MEW algorithm.

vertices of S_i . It iteratively adds the neighboring edge with the maximum weight to S_i until the capacity of S_i reaches C (lines 7–10). In order to balance the number of cross-committee transactions and the maintenance of the world state, the algorithm calculates the value of the segmentation function $F(S_i, v)$ for each vertex $v \in S_i$ before creating the next committee S_{i+1} . The function is defined as

$$F(S_i, v) = \sum_{u \in N(v) - S_i} (w(v, u) - \alpha \bar{w}). \quad (4)$$

Here, α represents the segmentation factor, with smaller values leading to more account segments per account. $\bar{w}$ denotes the average weight of edges in the account transaction graph. When $F(S_i, v) \geq 0$, it indicates a large weight $w(v, u)$, suggesting that the vertex v needs to be segmented to reduce cross-committee transactions. Hence, the edges connected to v and vertices in the account transaction graph are retained. Conversely, $F(S_i, v) < 0$ indicates a small weight $w(v, u)$. The vertex v will not be segmented to facilitate world state maintenance. In this case, these edges will be deleted from the graph and treated as cross-committee transactions. Algorithm 2 presents the procedure $\text{SegmentFunction}(S_i, \bar{w}, \alpha)$ for the above edge pruning based on (4).

Example 1: We use Fig. 1 to illustrate an example of the MEW algorithm. The graph consists of 10 vertices, with the total edge weight being 51 and an average weight of 3.64. We set $k = 2$, $\delta = 1.2$, and $\alpha = 1$, meaning each committee can accommodate six vertices, i.e., $C = 6$. According to the MEW algorithm, we select the edge (d, f) with the maximum weight and partitions it to committee S_1 , followed by edges (d, e) , (e, i) , (e, h) , (i, g) , (i, d) and (i, f) . At this point, committee S_1 has reached the capacity limit. Afterward, we calculate the value of the segmentation function for the vertices in committee S_1 . The value of vertex e is greater than 0; therefore, vertex e is segmented. Then we start to construct committee S_2 . We select the edge (b, a) with the maximum weight and partition it to committee S_2 , followed by edges (b, e) , (a, j) , (b, c) and (a, c) . In the sharding result, the cross-committee transactions are (b, h) and (j, g) , with a total weight of 4 and a segmented vertex of e .

C. OC Algorithm

The OC algorithm employs the idea of community discovery to divide the account transaction graph into committees. Each community represents a committee, with tight connections between accounts within the same committee and sparse connections between accounts in different committees, thereby

Algorithm 3: Initial Sharding Generation

Input: $G, k, \delta, \beta, LS_v$.
Output: $S = \{S_1, S_2, \dots, S_t\}$.

Procedure $\text{Sharding}(G, k, \delta, \beta, LS_v)$:

- 1 Sort vertices in V in descending order of vertex weights;
- 2 **while** $V \neq \emptyset$ **do**
- 3 Pick out the first member $v \in V$;
- 4 **foreach** $S_i \in LS_{N(v)}$ **do**
- 5 $B(S_i, v) = \frac{\sum_{u \in N(v)} w(v, u) \Gamma(S_i, u)}{w_v} \left(1 - \frac{|S_i|}{C}\right)$;
- 6 $B_{\max} = \text{Max}_i \{B(S_i, v)\}$;
- 7 **if** $B_{\max} == 0$ **then**
- 8 $LS_v \leftarrow S_j$ where $|S_j|$ is the minimum in S ;
- 9 **else**
- 10 **foreach** $S_i \in B_x(S_i, v)$ **do**
- 11 **if** $B_i/B_{\max} \geq \beta$ **then**
- 12 $LS_v \leftarrow LS_v \cup S_i$;
- 13 **end**
- 14 **return** $S = \{S_1, S_2, \dots, S_t\}$;

reducing cross-committee transactions. Community discovery approaches can be nonoverlapping or overlapping. In nonoverlapping communities, a node only belongs to one community, while in overlapping communities, it can belong to multiple communities simultaneously [36]. To adapt to state sharding, we adopt the idea of overlapping community discovery. However, since existing overlapping community discovery algorithms do not consider workload balance or the maintenance of the world state, we cannot apply them directly to our problem. The label propagation algorithm [37] is an efficient algorithm for overlapping community discovery, so we improve it for account partitioning.

Every vertex $v \in V$ is linked to a label set, denoted as LS_v , which includes the communities to which v is assigned. A label of v is a community in LS_v . We represent the belonging coefficient of v in S_i as $B(S_i, v)$. We define $B(S_i, v)$ by considering the connectivity between v and S_i , as well as the workload of S_i as follows:

$$B(S_i, v) = \frac{\sum_{u \in N(v)} w(v, u) \Gamma(S_i, u)}{w_v} \left(1 - \frac{|S_i|}{C}\right) \quad (5)$$

where $\Gamma(S_i, u)$ is set to 1 if the neighboring node $u \in N(v)$ belongs to the committee S_i and 0 otherwise. $1 - (|S_i|/C)$ is the penalty for workload. A large $|S_i|$ will lead to a small value of $B(S_i, v)$, i.e., if the committee S_i already has many accounts, the coefficient of v belonging to S_i will decrease. LS_v is updated in each iteration based on the value of $B(S_i, v)$.

The OC algorithm consists of two stages: 1) initial sharding generation and 2) committee merging. The algorithm constructs the initial committees based on the label propagation. Next, these initial committees are merged into k committees.

First Stage: The stage of initial sharding generation proceeds in iteration. Algorithm 3 outlines the pseudo-code for the procedure in the x th iteration. The algorithm prioritizes vertices with the highest weight (lines 2–4). For each vertex v , the belonging coefficient $B(S_i, v)$ is calculated for each $S_i \in LS_{N(v)}$. This involves considering the labels of neighboring nodes of v to update the label of v , a process known as label propagation (lines 5 and 6). $B_{\max}$ denotes the maximum value

Algorithm 4: Committee Merging

Input: $S = \{S_1, S_2, \dots, S_t\}$.
Output: $S = \{S_1, S_2, \dots, S_k\}$.

```

1 Procedure MergeCommittee( $S, k$ ):
2   while  $S \neq \emptyset$  do
3     Pick out the first member  $S_i \in S$  with smallest  $|S_i|$ ;
4     foreach  $S_j \in S \setminus S_i$  do
5        $M(S_i, S_j) = |\text{edge}(S_i, S_j)| |\text{vertex}(S_i, S_j)| \left(1 - \frac{|S_i| + |S_j|}{C}\right)$ ;
6      $S_{\max} = \text{argmax}_j \{M(S_i, S_j)\}$ ;
7      $S_{\max} \leftarrow S_i \cup S_{\max}$ ;
8     if  $|S| == k$  then
9       return  $S = \{S_1, S_2, \dots, S_k\}$ ;

```

of $B(S_i, v)$ for all $S_i \in LS_N(v)$ (line 7). If $B(S_i, v)$ is 0, the label of v is set to the committee with the minimum workload in S (lines 8 and 9). Otherwise, each $B(S_i, v)$ is compared with $B_{\max}$. If the division ratio is not less than the threshold β , it indicates that $B(S_i, v)$ is large, and the label S_i is added into LS_v (lines 11–13).

Suppose that this stage generates t committees. Recall that a load penalty is defined in the belonging coefficient function $B_x(S_i, v)$. The vertex with a belonging coefficient 0 has its label assigned to the least loaded committee (lines 8 and 9). This implies that t is greater than or equal to k . In the special case where t equals k , the initial sharding result is returned directly. Otherwise, these t committees must be merged so that the final number of committees is k .

Second Stage: Algorithm 4 outlines the pseudo-code for committee merging stage. The committees with a low workload are prioritized for merging (line 3). When a committee S_i is selected for merging, it will be combined with another committee S_j . We measure the performance of this merging operation using the merging coefficient $M(S_i, S_j)$ (line 5). The notation $|\text{edge}(S_i, S_j)|$ represents the number of cross-committee transactions between committees S_i and S_j , while $|\text{vertex}(S_i, S_j)|$ represents the number of vertices in both S_i and S_j . A high value of $|\text{edge}(S_i, S_j)|$ helps to reduce the number of cross-committee transactions, while a high value of $1 - (|S_i| + |S_j|)/C$ helps to reduce account segmentation. We refer to the workload penalty as $1 - (|S_i| + |S_j|)/C$ here. Then, the merging coefficient is

$$M(S_i, S_j) = |\text{edge}(S_i, S_j)| |\text{vertex}(S_i, S_j)| \left(1 - \frac{|S_i| + |S_j|}{C}\right). \quad (6)$$

S_i will be combined to the committee S_j with the maximum value of $M(S_i, S_j)$ (lines 6 and 7).

Example 2: Fig. 2 shows an example of the OC algorithm. The account transaction graph is the same as that in Fig. 1. We set $k = 2$, $\delta = 1.2$, and $\beta = 0.7$. The algorithm prioritally updates the label of the vertex with the maximum weight 20, i.e., vertex e ; as a result, $LS_e = (i, d, h)$. Next, the algorithm updates the label of vertex d whose vertex weight is 16, resulting in $LS_d = (d, i, f)$. We set the number of iterations to 5. The initial sharding stage generates three committees: 1) $S_1 = (h, b, e)$; 2) $S_2 = (b, a, c, j)$; and 3)

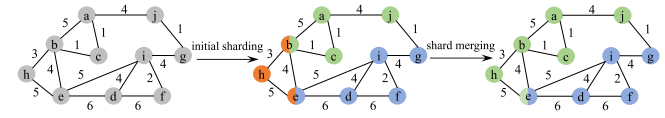


Fig. 2. Example of the OC algorithm.

$S_3 = (e, d, i, g, f)$. In the stage of committee merging, since S_1 only has three vertices, S_1 should be combined with committee S_2 or S_3 . Because $M(S_1, S_2) = 0$ and $M(S_1, S_3) = -(1/6)$, S_1 is combined with S_2 . As a result, the final result is: $S_1 = (h, b, e, a, c, j)$ and $S_3 = (e, d, i, g, f)$. There is one cross-committee transaction, represented by the edge (j, g) with a weight of 1. The vertex e exists in both committees.

D. Dynamic Sharding

Over time, the blockchain system experiences the inclusion of new transactions. These transactions may involve new accounts. We consider these new accounts and transactions as newly added vertices and edges in the account transaction graph G .

Definition 2 (Dynamic Account Transaction Graph): With the addition of these new vertices and edges, the graph $G = (V, E, W)$ is transformed into $G' = (V', E', W')$ where $V' = V \cup V_I$, $E' = E \cup E_I$, and $W' = W \cup W_I$. Here, V_I and E_I represent the set of newly added vertices and edges, and W_I is the set of weights of the newly added edges.

Let $(u', v') \in E_I$ be a new transaction. There are three possible scenarios for accounts u and v : 1) both accounts are new, i.e., $u' \in V_I \wedge v' \in V_I$; 2) only one account is new, and the other is existing, i.e., $u' \in V \wedge v' \in V_I$ or $u' \in V_I \wedge v' \in V$; and 3) both accounts are existing, i.e., $u' \in V \wedge v' \in V$. We assign u' and v' to the same committee to reduce the amount of cross-committee transactions. Specifically, for the first case, we place both accounts of the transaction into the least loaded committee to maintain workload balancing among the committees. For the second case, the existing account may exist in multiple committees, so we carefully select a committee from them and place the new account into it. In the third case, if both accounts exist in the same committee, only the edge weight needs to be updated; otherwise, we select a committee to store both accounts.

Motivated by the linear deterministic greedy (LDG) algorithm [2], given a vertex v , we can determine the committee of v based on a scoring function: $G(v, S_i) = |S_i \cap N(v)| (1 - |S_i|/C)$, where $|S_i \cap N(v)|$ represents the number of neighboring vertices of v in S_i . According to this function, adjacent vertices will be partitioned into the same committee to produce fewer cross-committee edges. To integrate edge weights, we change the scoring function to

$$G(v, S_i) = \sum_{u \in S_i \cap N(v)} w(v, u) \left(1 - \frac{|S_i|}{C}\right) \quad (7)$$

where $\sum_{u \in S_i \cap N(v)} w(v, u)$ represents the total weight of edges connecting vertex v and committee S_i .

Algorithm 5 outlines the procedure for the dynamic sharding. In the first case, the new account is allocated to the

Algorithm 5: Dynamic Account Partitioning

Input: $G', S, (u', v')$.
Output: $S = \{S_1, S_2, \dots, S_k\}$.

```

1 if  $v' \in V_I \& u' \in V_I$  then
2    $S_i \leftarrow$  the committee with the minimum  $|S_i|$  in  $S$ ;
3    $S_i \leftarrow S_i \cup u', S_i \leftarrow S_i \cup v'$ ;
4 else if  $v' \in V_I \& u' \notin V_I$  then
5    $S_i = \arg \max_{S_i \in S(u')} \left( \sum_{s \in S_i \cap N(v')} w(v', s) \left( 1 - \frac{|S_i|}{C} \right) \right)$ ;
6    $S_i \leftarrow S_i \cup v'$ ;
7 else
8   if  $u'$  and  $v'$  are in the same committee then
9     return  $S$ ;
10  else
11     $S_i = \arg \max_{S_i \in S(v') \cup S(u')} (G(v', S_i) + G(u', S_i))$ ;
12     $S_i \leftarrow S_i \cup u', S_i \leftarrow S_i \cup v'$ ;

```

least loaded committee (lines 1–3). In the second case, for the new account v' , the algorithm computes the score $G(v', S_i)$ for all $S_i \in S(u')$. Here, $S(u')$ is the set of committees that own the state segments of u' . The algorithm will allocate v' to a committee in $S(u')$ to decrease state segmentation of v' . Specifically, it chooses the committee $S_i = \arg \max_{S_i \in S(u')} G(v', S_i)$ (lines 4–7). For the third case, if there is a committee that owns the state segments of both accounts, the algorithm will not modify the initial sharding (lines 8 and 9). Otherwise, the algorithm will select a committee S_i in $S(v') \cup S(u')$ to store accounts (lines 11 and 12). The total score $G(v', S_i) + G(u', S_i)$ is used here. If $u' \in S_i$, v' is added to S_i as a state segment and vice visa.

E. Account Management

We use a state tree to manage the state segmentation. Each committee only keeps track of the account information within that committee. Fig. 3 presents an example of the state tree within a committee. The state of account v is denoted as $S_v = \{A_v | T_v, b, \eta, \mu\}$. Here, A_v represents the address of v . T_v is a vector in the form of $T_v = \{S_m, \{S_1, S_2, \dots, S_k\}\}$ that records the IDs of committees where the segmented state exists. S_m is the master account that maintains the world state of v . Let b represent v 's balance in the current committee. η is the nonce, and μ denotes the account type with "1" indicating an external account and "0" expressing a contract account. Since all committees store S_v for all accounts, each committee is aware of the state segmentation for any account.

V. EXPERIMENTAL EVALUATION**A. Experimental Setup**

Datasets: We conducted our experiments using real Ethereum transaction data and randomly selected 30 000 transactions from 6949 accounts as of May 2023. The sharded blockchain network comprises 1000 nodes, with 1/3 being malicious nodes randomly distributed in the sharded network. The reputation score of each node is initially set to 5. In each epoch, each malicious node has a 40% probability of engaging in malicious behavior. We increased the number of committees from 16 to 128 to test the scalability of the

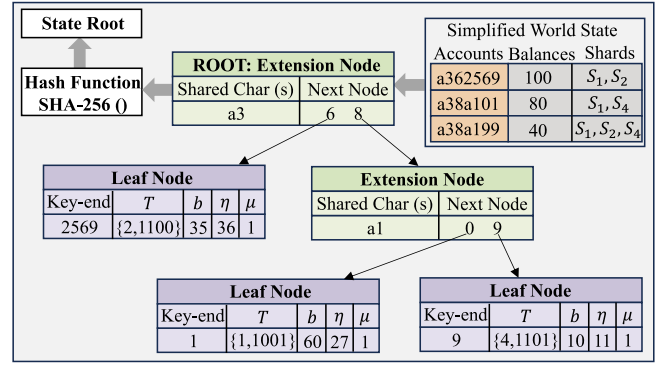


Fig. 3. State tree.

proposed approaches [14], [22], [23], [33]. Among the 30 000 transactions, we randomly selected 5000, 10 000, 15 000, 20 000, and 30 000 transactions. The balancing factor δ is 1.2, the segmentation factor α is 0.8, and the threshold β is 0.9. We also utilized a blockchain sharding simulator¹ to evaluate system throughput and transaction latency. The simulator is capable of generating transactions, packaging them, and creating blocks, each capable of handling 2000 transactions. The simulator uses a function to choose malicious nodes and then simulates malicious behaviors.

Competitors: In network sharding, we compared our reputation score-based approach (Algorithm 1) with random sharding. In account partitioning, we compared six algorithms: 1) the MEW algorithm (Algorithm 2); 2) the OC algorithm (Algorithms 3 and 4); 3) random partitioning; 4) the Metis algorithm; 5) the Monoxide algorithm [22]; and 6) the dynamic sharding algorithm (Algorithm 5).

Metrics: In network sharding, we measured two metrics: 1) the reputation scores of nodes and 2) the number of malicious nodes in each committee. In account partitioning, we measured five metrics: 1) the total weight of cross-committee transactions; 2) the number of account segments; 3) workload balancing; 4) system throughput; and 5) transaction latency. The workload balancing is measured by the difference between the maximum and the minimum number of accounts in a committee. In dynamic sharding, we measured two metrics: 1) the number of reallocated accounts and 2) the total weight of cross-committee transactions. We evaluated the system scalability by measuring the throughput and transaction processing latency with different numbers of committees [14], [22], [23].

B. Performance of Network Sharding Algorithms

Fig. 4 displays the reputation scores of our proposed approach at different epochs. During each epoch, the reputation scores of 1000 nodes are sorted in ascending order. The growth of node reputation scores remains consistent across different epochs. This stability is because the reputation score function incorporates the logarithmic computation of the standard deviation σ , which reflects the variations in reputation scores among all nodes [refer to (1)]. When there is a substantial difference in node reputation scores, the growth

¹<https://github.com/samwakewatson/MSBS>

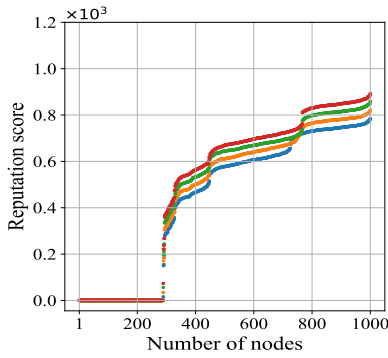


Fig. 4. Node reputation scores at different epochs.

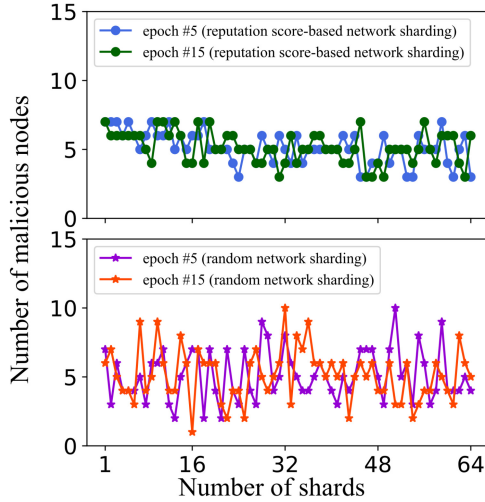


Fig. 5. Number of malicious nodes.

of the reputation score for high-reputation nodes is constrained to prevent these nodes from dominating the sharding process. It is important to note that the first 300 nodes have a reputation score of 0, indicating that these nodes are malicious. We penalize malicious behavior by reducing the reputation score of a malicious node. This mitigates the possibility of a malicious node attaining a high-reputation score, and consequently, it will not be selected as a leader node. This effectively enhances the security of the sharding system.

Fig. 5 compares the reputation score-based approach with random network sharding. It plots the number of malicious nodes as a function of the committee amount at epoch #5 and epoch #15, respectively. The above subfigure is the performance of the reputation score-based approach; the below subfigure is the performance of the random network sharding. The number of committees varies from 1 to 64. Ideally, malicious nodes are uniformly distributed among different committees, i.e., the number of malicious nodes is equal in different committees. For the security of the sharding system, malicious nodes cannot be concentrated in particular committees. From the figure, we can see that malicious nodes distribute more evenly for the reputation score-based approach regardless of the number of committees. We compute the difference between the maximum value and the minimum value of the number of malicious nodes. In epoch #5, the

difference is 4 for the reputation score-based approach, while this is 8 for the random network sharding. There is a 50% reduction.

C. Performance of Account Partitioning

1) *Total Weight of Cross-Committee Transactions:* Fig. 6(c) illustrates the impact of the number of transactions on systems employing 16, 32, 64, and 128 committees, respectively. The OC algorithm generally outperforms the other algorithms, while the random partitioning algorithm performs the worst among the four algorithms. As the number of transactions increases from 5000 to 30 000, the total weight of cross-committee transactions grows for all four algorithms. However, the growth rate of cross-committee transaction weights is lower for the OC and MEW algorithms. This demonstrates the strength of the OC and MEW algorithms, particularly with a larger number of transactions. Furthermore, the total weight of cross-committee transactions increases as the network incorporates more committees for all approaches. At 128 committees and a transaction count of 30 000, the OC and MEW algorithms reduce cross-committee transaction weights by 53.3% and 34.8%, respectively, compared to the Metis algorithm. This highlights the superiority of the OC algorithm.

The segmentation factor α affects whether the account is segmented [see (4)], and the threshold β affects the retention of labels (line 12, Algorithm 3). Fig. 7(a) and (b) show the total weight of cross-committee transactions generated by the MEW algorithm for α of 0.7, 0.8, and 0.9 under the number of committees of 32 and 64. As α decreases from 0.9 to 0.7, the total weight decreases. Similarly, Fig. 7(c) and (d) show the effect when β is 0.85, 0.9, and 0.95. Again, as β decreases, the total weight of cross-committee transactions decreases. This is because the smaller the α , the more account segments there are, which reduces the cross-committee transaction amount. The reason is the same for the effect of β .

2) *Number of Account Segments:* Fig. 8 shows the number of account segments when the transaction amount grows from 5000 to 30 000. The number of committees is 16, 32, 64, and 128, respectively. It can be observed that the number of account segments for MEW and OC algorithms is less than that of the random partitioning algorithm. The performances of MEW and OC algorithms are comparable in most cases. When there are 30 000 transactions to store in 64 or 128 committees, the MEW algorithm performs much better than the OC algorithm, which has the number of account segments about 10 times larger. This is because $|\text{edge}(S_i, S_j)|$ occupies a large portion during the committee merging; as a result, the total cross-committee transaction weight has significantly decreased as shown in Fig. 6(a) and (b).

3) *Workload Balancing:* Workload balancing in the sharded blockchain refers to a relatively balanced distribution of accounts among committees, preventing some committees from becoming hot due to the high volume of transactions being handled. Good workload balancing improves system performance. Fig. 9 shows the performance of workload

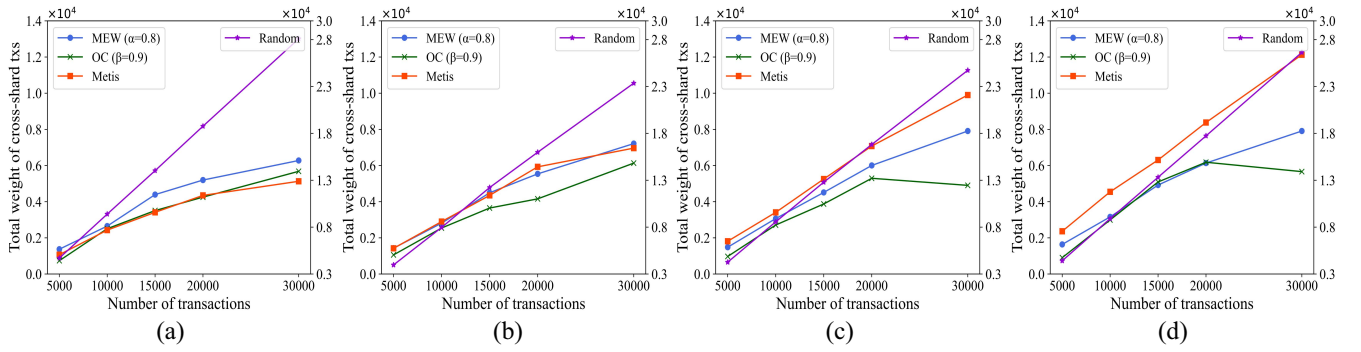


Fig. 6. Total weight of cross-committee transactions. (a) 16 committees. (b) 32 committees. (c) 64 committees. (d) 128 committees.

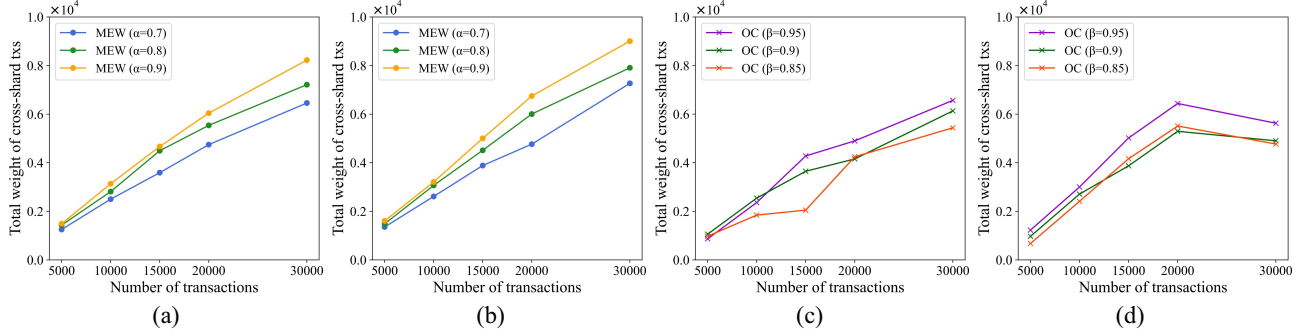


Fig. 7. Total weight of cross-committee transactions (MEW algorithm). (a) 32 committees. (b) 64 committees. (c) 32 committees. (d) 64 committees.

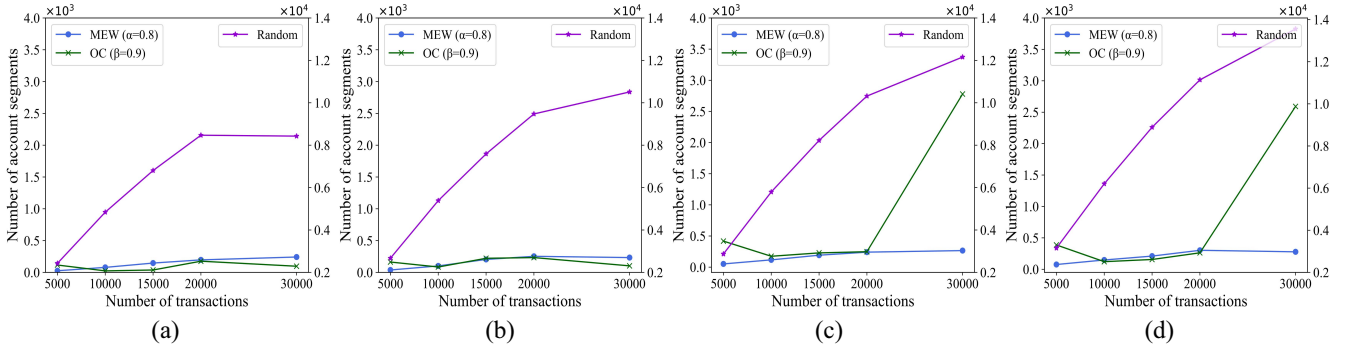


Fig. 8. Number of account segments. (a) 16 committees. (b) 32 committees. (c) 64 committees. (d) 128 committees.

balancing for all algorithms. At 16 and 32 committees, the random partitioning algorithm outperforms the OC approach. This is because, during the committee merging, the workload of the merged committee becomes larger, resulting in a larger difference from the least loaded committee. The MEW method surpasses the Metis algorithm, but the random partitioning algorithm is the most effective. However, as the number of committees rises, MEW and OC perform better.

4) *Dynamic Sharding*: For the case of new transactions in the blockchain, we propose the dynamic sharding algorithm to update the original sharding result, e.g., by increasing the account segments. We first used the MEW algorithm to generate the original sharding result with 32 committees and 20000 transactions. The amount of new transactions is 1000, 2000, 3000, and 4000, respectively. We partitioned new transactions based on the random partitioning algorithm, the Metis algorithm, and the dynamic sharding algorithm. We

measured the number of reallocated accounts and the total weight of cross-committee transactions. Fig. 10(a) illustrates the number of reallocated accounts. It can be observed that the Metis algorithm has the maximum number of reallocated accounts, as it requires resharding the entire account transaction graph. Fig. 10(b) illustrates the total weight of cross-committee transactions. The dynamic sharding algorithm performs the best, producing the minimum sum of cross-committee transaction weights. When both accounts of the new transaction exist, the accounts are partitioned to the committee with the maximum score, which reduces the cross-committee transaction weights. Specifically, the total weight of the dynamic sharding algorithm is 68.8% that of the Metis algorithm when the number of new transactions is 4000.

5) *Throughput and Latency*: We vary the transaction arrival rate and the number of committees. In Fig. 11, the transaction arrival rate is set at 4000 transactions per second,

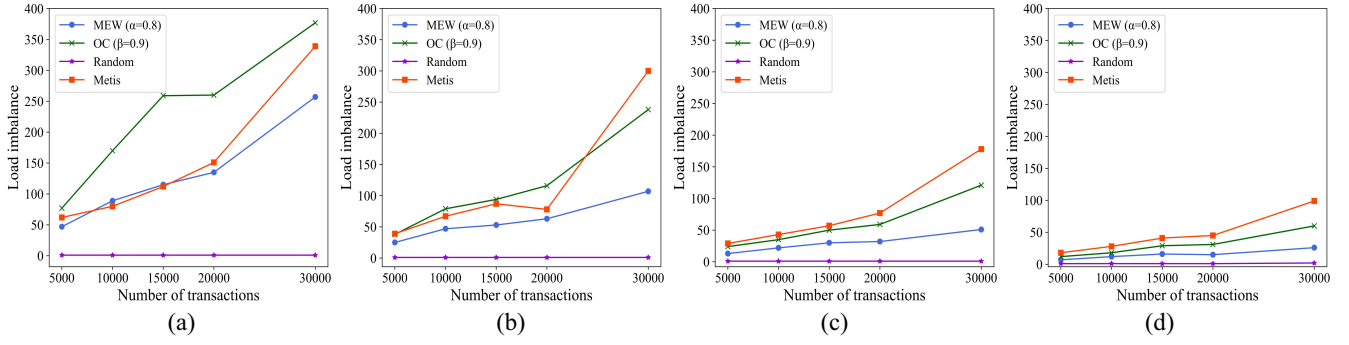


Fig. 9. Load imbalance. (a) 16 committees. (b) 32 committees. (c) 64 committees. (d) 128 committees.

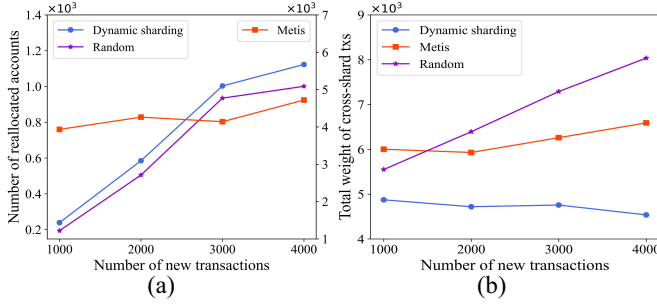


Fig. 10. Performance of dynamic sharding. (a) Number of reallocated accounts. (b) Cross-committee Tx weights.

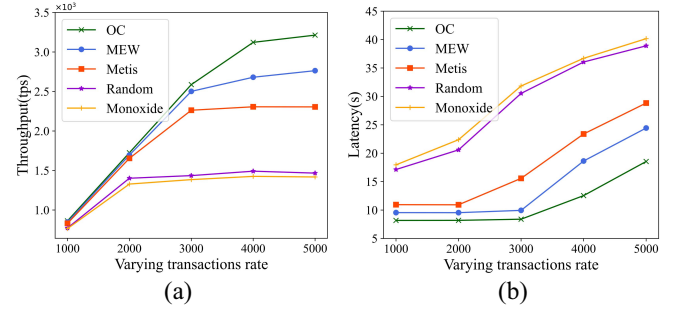


Fig. 12. System throughput and transaction latency (32 committees). (a) System throughput. (b) Transaction latency.

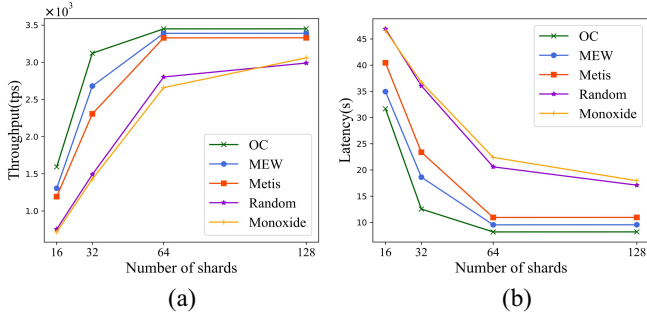


Fig. 11. System throughput and transaction latency (TX arrival rate=4000 TXs/s). (a) System throughput. (b) Transaction latency.

while we vary the number of committees from 16 to 128. Fig. 11(a) depicts the system throughput, showing that the OC algorithm achieved the highest throughput, followed by the MEW algorithm. Both algorithms outperform Metis, random partitioning, and Monoxide algorithms. Fig. 11(b) illustrates the transaction latency, with the OC and MEW algorithms exhibiting lower transaction latency, which decreased as the number of committees increased. The high throughput and low latency of the OC and MEW algorithms can be attributed to the reduction in cross-committee transactions, leading to decreased intercommittee communication overheads. From the experiments, the system throughput increases with the number of committees. The increase is almost linear when the number of committees grows from 16 to 64. These experiments demonstrate the scalability of NSshard as the number of committees increases.

Next, we fix the number of committees at 32 and vary the transaction arrival rate, with the results shown in Fig. 12. The transaction arrival rate ranges from 1000 transactions per second to 5000 transactions per second. Once again, the OC algorithm demonstrates the highest throughput and lowest latency, followed by the MEW algorithm. They perform better than Monoxide. As the transaction arrival rate increased, the throughput of all algorithms was gradually saturated, and the latency increased. It can be seen that our MEW and OC algorithms achieve higher throughput and lower latency than the comparison methods in all cases in Figs. 11 and 12.

VI. CONCLUSION

The sharding technique divides the blockchain into several committees with a consequent reduction of nodes in each committee. This allows malicious nodes to attack only specific committees, leading to a decrease in system security. At the same time, frequent cross-committee transactions also seriously affect the performance of the blockchain system. In this article, we have proposed a new blockchain sharding framework to improve system security and reduce cross-committee transactions. We have proposed the reputation score-based network sharding that calculates the reputation score based on the historical behavior of each node. We have also proposed an account partitioning scheme to support state sharding. We have designed the MEW and the OC algorithms to minimize the amount of cross-committee transactions while considering both workload balance and state maintenance. Experimental results on Ethereum transaction data have shown that compared to the Metis algorithm, our MEW algorithm and

BOCD algorithm reduce 53.3% and 34.8% of cross-committee transactions at 128 committees, respectively. Our dynamic sharding algorithm only produces 68.8% of the total cross-committee transactions of the Metis algorithm.

Our network sharding approach elects leader nodes based on their reputation scores. However, malicious nodes may promote their scores through disguised behavior; as a result, they may launch attacks or manipulate transactions to disrupt system security. Therefore, our future research intends to integrate security techniques, e.g., malicious node detection, into the scoring method by monitoring node behaviors, making it difficult for malicious nodes to disguise themselves. Moreover, although our account partitioning algorithms generate fewer cross-committee transactions, they are still inevitable. We will investigate other techniques, e.g., cross-committee protocols and relay transactions, to reduce the confirmation latency of cross-committee transactions.

REFERENCES

- [1] C. Xu, Y. Qu, T. H. Luan, P. W. Eklund, Y. Xiang, and L. Gao, "A lightweight and attack-proof bidirectional blockchain paradigm for Internet of Things," *IEEE Internet Things J.*, vol. 9, no. 6, pp. 4371–4384, Mar. 2022.
- [2] C. Zhang et al., "A blockchain-based model migration approach for secure and sustainable federated learning in IoT systems," *IEEE Internet Things J.*, vol. 10, no. 8, pp. 6574–6585, Apr. 2023.
- [3] Q. Wang, X. Zhu, Y. Ni, L. Gu, and H. Zhu, "Blockchain for the IoT and industrial IoT: A review," *Internet Things*, vol. 10, Jun. 2020, Art. no. 100081.
- [4] R. Li, T. Song, B. Mei, H. Li, X. Cheng, and L. Sun, "Blockchain for large-scale Internet of Things data storage and protection," *IEEE Trans. Services Comput.*, vol. 12, no. 5, pp. 762–771, Sep./Oct. 2019.
- [5] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system." 2009. [Online]. Available: <https://metzdowd.com>
- [6] B. Wang et al., "BLOCKKEY: Hunting for DeFi attacks on blockchain," in *Proc. 43rd Int. Conf. Softw. Eng., Companion Proc.*, 2021, pp. 17–20.
- [7] Y. Yan et al., "Confidentiality support over financial grade consortium blockchain," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2020, pp. 2227–2240.
- [8] S. Kumar, R. S. Rathore, U. Dohare, O. Kaiwartya, J. Lloret, and N. Kumar, "BEET: Blockchain enabled energy trading for e-mobility oriented electric vehicles," *IEEE Trans. Mobile Comput.*, vol. 23, no. 4, pp. 3018–3034, Apr. 2024.
- [9] K. Park, J. Lee, A. K. Das, and Y. Park, "BPPS: Blockchain-enabled privacy-preserving scheme for demand-response management in smart grid environments," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 2, pp. 1719–1729, Mar./Apr. 2023.
- [10] F. Hashim, K. Shuaib, and F. Sallabi, "MedShard: Electronic health record sharing using blockchain sharding," *Sustainability*, vol. 13, no. 11, p. 5889, 2021.
- [11] C.-H. Lin, L.-C. Huang, S.-C. T. Chou, C.-H. Liu, H.-F. Cheng, and I.-J. Chiang, "Temporal event tracing on big healthcare data analytics," in *Proc. IEEE Int. Congr. Big Data*, 2014, pp. 281–287.
- [12] J. Zhang, Z. Hong, X. Qiu, Y. Zhan, S. Guo, and W. Chen, "SkyChain: A deep reinforcement learning-empowered dynamic blockchain sharding system," in *Proc. 49th Int. Conf. Parallel Process.*, 2020, pp. 1–11.
- [13] J. C. Corbett et al., "Spanner: Google's globally distributed database," *ACM Trans. Comput. Syst.*, vol. 31, no. 3, pp. 1–22, 2013.
- [14] L. N. Nguyen, T. D. T. Nguyen, T. N. Dinh, and M. T. Thai, "OptChain: Optimal transactions placement for scalable blockchain sharding," in *Proc. IEEE 39th Int. Conf. Distrib. Comput. Syst.*, 2019, pp. 525–535.
- [15] Y. Liu et al., "Building blocks of sharding blockchain systems: Concepts, approaches, and open problems," *Comput. Sci. Rev.*, vol. 46, Nov. 2022, Art. no. 100513.
- [16] E. Kokoris-Kogias, P. Jovanovic, L. Gasser, N. Gailly, E. Syta, and B. Ford, "OmniLedger: A secure, scale-out, decentralized ledger via sharding," in *Proc. IEEE Symp. Security Privacy*, 2018, pp. 583–598.
- [17] E. Syta et al., "Scalable bias-resistant distributed randomness," in *Proc. IEEE Symp. Security Privacy*, 2017, pp. 444–460.
- [18] S. Micali, M. Rabin, and S. Vadhan, "Verifiable random functions," in *Proc. 40th Annu. Symp. Found. Comput. Sci.*, 1999, pp. 120–130.
- [19] L. Luu, V. Narayanan, C. Zheng, K. Baweja, S. Gilbert, and P. Saxena, "A secure sharding protocol for open blockchains," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2016, pp. 17–30.
- [20] M. Zamani, M. Movahedi, and M. Raykova, "RapidChain: Scaling blockchain via full sharding," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 931–948.
- [21] G. Wang, Z. J. Shi, M. Nixon, and S. Han, "SoK: Sharding on blockchain," in *Proc. 1st ACM Conf. Adv. Financ. Technol.*, 2019, pp. 41–61.
- [22] J. Wang and H. Wang, "Monoxide: Scale out blockchain with asynchronous consensus zones," in *Proc. 16th USENIX Conf. Netw. Syst. Design Implement.*, 2019, pp. 95–112.
- [23] H. Huang et al., "BrokerChain: A cross-shard blockchain protocol for account/balance-based state sharding," in *Proc. IEEE Conf. Comput. Commun.*, 2022, pp. 1968–1977.
- [24] L. Jia, Y. Liu, K. Wang, and Y. Sun, "Estuary: A low cross-shard blockchain sharding protocol based on state splitting," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 3, pp. 405–420, Mar. 2024.
- [25] G. Karypis and V. Kumar, "A fast and high quality multilevel scheme for partitioning irregular graphs," *SIAM J. Sci. Comput.*, vol. 20, no. 1, pp. 359–392, 1998.
- [26] H. Chen and Y. Wang, "SSChain: A full sharding protocol for public blockchain without data migration overhead," *Pervasive Mobile Comput.*, vol. 59, Oct. 2019, Art. no. 101055.
- [27] Z. Hong, S. Guo, E. Zhou, W. Chen, H. Huang, and A. Zomaya, "GriDB: Scaling blockchain database via sharding and off-chain cross-shard mechanism," *Proc. VLDB Endow.*, vol. 16, no. 7, pp. 1685–1698, 2023.
- [28] M. Li, W. Wang, and J. Zhang, "LB-chain: Load-balanced and low-latency blockchain sharding via account migration," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 10, pp. 2797–2810, Oct. 2023.
- [29] Z. Hong, S. Guo, P. Li, and W. Chen, "Pyramid: A layered sharding blockchain system," in *Proc. IEEE Conf. Comput. Commun.*, 2021, pp. 1–10.
- [30] A. Liu et al., "CHERUBIM: A secure and highly parallel cross-shard consensus using quadruple pipelined two-phase commit for sharding blockchains," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 3178–3193, 2024.
- [31] M. Li, X. Luo, K. Xue, Y. Xue, W. Sun, and J. Li, "A secure and efficient blockchain sharding scheme via hybrid consensus and dynamic management," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 5911–5924, 2024.
- [32] Y. Zhang, S. Pan, and J. Yu, "TxAllo: Dynamic transaction allocation in sharded blockchain systems," in *Proc. IEEE 39th Int. Conf. Data Eng.*, 2023, pp. 721–733.
- [33] J. Xu, Y. Ming, Z. Wu, C. Wang, and X. Jia, "X-shard: Optimistic cross-shard transaction processing for sharding-based blockchains," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 4, pp. 548–559, Apr. 2024.
- [34] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *J. Stat. Mech., Theory Exp.*, vol. 2008, no. 10, 2008, Art. no. P10008.
- [35] P. Zhang, W. Guo, Z. Liu, M. Zhou, B. Huang, and K. Sedraoui, "Optimized blockchain sharding model based on node trust and allocation," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 3, pp. 2804–2816, Sep. 2023.
- [36] J. Yang and J. Leskovec, "Overlapping community detection at scale: A nonnegative matrix factorization approach," in *Proc. 6th ACM Int. Conf. Web Search Data Min.*, 2013, pp. 587–596.
- [37] S. Gregory, "Finding overlapping communities in networks by label propagation," *New J. Phys.*, vol. 12, no. 10, 2010, Art. no. 103018.